The Journal of Open Source Software

CausalTables.jl: Simulating and storing data for statistical causal inference in Julia

Salvador V. Balkus [1¶] **and Nima S. Hejazi** [1]

1 Department of Biostatistics, Harvard T.H. Chan School of Public Health ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Oskar Laverny

Reviewers:

- @juliohm
- @olivierlabayle

Submitted: 26 November 2024
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

Estimating the strength of causal relationships between variables is an important problem across many scientific disciplines. A variety of statistical methods have been developed to estimate and obtain inferences about causal quantities, yet few tools readily support the comparison of candidate approaches. `CausalTables.jl` offers tools to evaluate and compare statistical causal inference methods in Julia. The package provides two main functionalities. Firstly, it implements a `CausalTable` interface for storing data with partially-labeled causal structure in a `Tables.jl`-compatible format. Secondly, it introduces a `StructuralCausalModel` for randomly generating data with a user-specified causal structure and computing ground truth parameters under the given experiment. Together, these functionalities expand the Julia ecosystem by supporting the use and benchmarking of the growing number of causal inference methods.

Statement of need

The quantitative science of causal inference has emerged over the past three decades as a set of formalisms for studying cause-and-effect relationships between variables from observed data (Hernán & Robins, 2020; Pearl, 2009); causal inference techniques have helped applied scientists and decision-makers better understand important phenomena in fields ranging from health and medicine to politics and economics. As interest in causal inference has grown across many disciplines, so too has the development of software tools for estimating causal effects. While Julia packages for causal inference have begun to emerge—with examples including, for estimation, `TMLE.jl` (Labayle et al., 2024) and `CausalELM.jl` (Colby, 2024), and, for causal discovery, `CausalInference.jl` (Schauer et al., 2024)—the ecosystem is still in its infancy. New methods for causal inference are being developed at a rapid pace, underscoring the need for tools designed to support the evaluation and comparison of their performance. `CausalTables.jl` aims to provide such a tool for the Julia language. Currently, attempts to benchmark causal inference methods in Julia face two major challenges.

First, packages often have inconsistent APIs. For example, some packages require the user to provide treatment and response variables as individual vectors, while others require the entire dataset in a `Tables.jl`-compliant format, with treatment and response variables labeled via strings or symbols. `CausalTables.jl` provides a `CausalTable` interface that simplifies packaging the data and auxiliary causal knowledge together. Second, benchmarking of methods requires simulating data for numerical experiments from a Structural Causal Model (SCM) (Pearl, 2009) so as to compare candidate estimators to an underlying ground truth (encoded via interventions on the SCM). An SCM defines causal structure by envisaging a data-generating process as random draws from a sequence of non-parametric structural equations, with each draw depending on realizations from draws preceding it. `CausalTables.jl` provides a simple, user-friendly way to define an SCM, sample data randomly from it, and compute or approximate the underlying true values of several common causal effect parameters.

By addressing these two major challenges, `CausalTables.jl` simplifies and accelerates the development of tools for statistical causal inference in Julia. The `CausalTable` interface extends `Tables.jl`, the most common interface for accessing tabular data in Julia (Quinn et al., 2024). The SCM framework operates in conjunction with `Distributions.jl`, the premier Julia package for working with random variables (Besançon et al., 2021; Lin et al., 2019). By integrating seamlessly with other commonly used packages in the Julia ecosystem, `CausalTables.jl` ensures both compatibility and ease of use for statisticians and applied scientists alike.

Instructional use cases

A standard causal inference problem is to estimate the effect of a treatment variable A on a response variable Y in the presence of confounders W. One can benchmark causal inference methods in two ways: either by imposing a causal structure on an existing dataset, or by drawing new data randomly from a programmatically-defined SCM.

Wrapping an existing dataset with causal structure is easy. The `CausalTable` constructor creates a `Tables.jl`-compliant data structure that wraps any existing data that already satisfies the `Tables.jl` interface with additional information about its causal structure. Calling convenience functions on this object allows users to perform data processing tasks common in causal inference, such as selecting or intervening on specific variables. For example, the `parents` function can be used to select only variables denoted as causes of another given variable.

```julia
using CausalTables

# Example data in a Tables-compatible format
tbl = (W = [0.2, 0.4, 0.7],
       A = [false, true, true],
       Y = [0.8, 1.2, 2.3])

# Wrap the data as a CausalTable
ct_wrap = CausalTable(tbl; treatment = :A, response = :Y, confounders = [:W])

# Select only variables upstream from the response
parents(ct_wrap, :Y)
```

```
CausalTable
┌────────┬───────┐
│      W │     A │
│ Float64 │  Bool │
├────────┼───────┤
│  0.200 │ false │
│  0.400 │  true │
│  0.700 │  true │
└────────┴───────┘
Summaries: NamedTuple()
Arrays: NamedTuple()
```

Simulating causal data for different settings is slightly more involved. In the remainder of this section, we will present two example use cases of how `CausalTables.jl` can be used as a benchmarking tool.

DRAFT

Example 1: Average Treatment Effect

The prototypical causal inference problem involves estimating the average treatment effect (ATE) of a binary treatment A. The ATE describes the difference in the counterfactual mean of Y had every unit been treated versus no unit treated. An example SCM describing this scenario might be the following:

$$
\begin{aligned}
W &\sim Beta(2, 4) \\
A &\sim Bernoulli(W) \\
Y &\sim Normal(A + W, 1)
\end{aligned}
$$

To compute the ground truth ATE via `CausalTables.jl`, we define the SCM above by enumerating the sequence of random variables to be drawn using the `@dgp` macro. Then, we create a `StructuralCausalModel` object which labels the steps we want to consider as treatment, response, and confounders. Finally, we randomly draw datasets from the newly instantiated `StructuralCausalModel` using the `rand` function.

```julia
using Distributions

# Define the sequence of random variables to be drawn
dgp = @dgp(
    W ~ Beta(2, 4),
    A ~ Bernoulli.(0.5 .* W .+ 0.2),
    Y ~ Normal.(W .+ A, 1)
)

# Create a structural causal model (SCM) from the DGP
scm = StructuralCausalModel(dgp;
  treatment = :A, response = :Y, confounders = [:W]
)

ct = rand(scm, 500) # randomly draw from the SCM
```

`CausalTables.jl` provides functions to approximate ground truth values of common causal estimands such as the ATE (using `ate`). These functions also simultaneously estimate the corresponding efficiency bound—the asymptotic lower bound on the variance—for a class of estimators (those that are regular and asymptotically linear) commonly used in causal inference. This facilitates the comparison of candidate estimators not only in terms of their bias (average distance from the ground truth) but also their efficiency. Below, we demonstrate these for the example SCM given above.

```julia
ate(scm) # average treatment effect
```

```
(μ = 1.002, eff_bound = 2.001)
```

In addition, `CausalTables.jl` provides a low-level interface allowing users to (1) apply common interventions to the treatment variable in a `CausalTable`, and (2) compute ground truth conditional densities and functions of these (e.g., mean, variance), which typically arise as nuisance parameters in the construction of estimators in causal inference. For example, below, we compute the difference in the conditional mean of Y under treatment versus no treatment, the difference of which is the ATE.

```julia
treated = intervene(ct, treat_all)    # CausalTable with everyone treated
untreated = intervene(ct, treat_none) # CausalTable with no one treated
mean(conmean(scm, treated, :Y) .- conmean(scm, untreated, :Y))
```

```
1.000
```

The above recovers an estimate of the ground truth via plug-in estimates based on the outcome regression (i.e., the conditional expectation of the outcome, given treatment and covariates).

DRAFT

Alternatively, one can also compute an inverse probability weighted (IPW) estimate with ground truth weights using the `propensity` function:

```
y = responsematrix(ct) # get the response
a = treatmentmatrix(ct) # get the treatment
mean(y .* (2 * a .- 1) ./ propensity(scm, ct, :A))
```

```
1.021
```

Finally, as an alternative, one can randomly generate a new counterfactual response value for each unit in a `CausalTable` under a given intervention using `draw_counterfactual`, and then compute the ATE directly:

```
y_treated = draw_counterfactual(scm, ct, treat_all)
y_untreated = draw_counterfactual(scm, ct, treat_none)
mean(y_treated .- y_untreated)
```

```
0.994
```

Example 2: Modified Treatment Policies

`CausalTables.jl` is not limited to settings with binary treatments; it also supports other estimands. Consider the following SCM, in which the treatment A is continuous-valued.

```
dgp = @dgp(
    W1 ~ Poisson(10),
    W2 ~ Bernoulli(0.5),
    A ~ (@. Normal(W1 + 5*W2, 1)),
    Y ~ (@. Normal(2*A + W2*A + 0.5*W1, 1 + W2))
)

scm = StructuralCausalModel(dgp;
  treatment = :A, response = :Y, confounders = [:W1, :W2]
)
```

In the continuous treatment setting, a common causal estimand is the effect of a *modified treatment policy* (MTP), which corresponds to the question: "how would the counterfactual outcome change had an intervention $d(a, w; \delta)$ been applied to the observed treatment a?" (Haneuse & Rotnitzky, 2013). In this setting, rather than estimating an ATE, we estimate an *average policy effect* (APE)—the difference between Y under the natural treatment value a and the intervened-upon treatment that results from the MTP $d(a, w; \delta)$. A common example is the additive MTP $d(a, w; \delta) = a + \delta$; when the relationship between A and Y is known to be linear, this is equivalent to the slope in a linear regression model, but, using `CausalTables.jl`, we can approximate the ground truth APE even when the relationship is nonlinear. We demonstrate this below for an MTP indexed by the choice $\delta = 1$.

```
ape(scm, additive_mtp(1)) # average policy effect
```

```
(μ = 2.502, eff_bound = 5.242)
```

One strategy for estimating an APE is to fit a parametric outcome regression model and use it to predict the outcome Y under the modified treatment policy $d(A, W; \delta) : A \to A + \delta$; the average difference of these predictions on the observed data yields the APE. We can use `CausalTables.jl` to simulate the output of such a procedure had we known the true value of the outcome regression:

```
ct = rand(scm, 500)  # Randomly draw data
ct_intervened = intervene(ct, additive_mtp(1))  # apply MTP
mean(conmean(scm, ct_intervened, :Y) .- responsematrix(ct))
```

```
2.529
```

DRAFT

Closing remarks

The flexibility of `CausalTables.jl` allows users to easily extract ground truth values for any relevant aspect of a data-generating process. This supports benchmarking causal estimators of virtually any estimand that fits in the SCM framework, not just those mentioned in these two examples. While the package includes high-level functions to approximate several common causal estimands, users can also write their own interventions and use low-level functions such as `intervene`, `draw_counterfactual`, and `condensity` to approximate the ground truth of new causal estimands. Thus, `CausalTables.jl` serves as a useful tool for scientists seeking to evaluate and benchmark various causal inference methods in simulation studies.

Acknowledgements

Salvador Balkus acknowledges support from the National Institute of Environmental Health Sciences (award no. T32 ES007142) and the National Science Foundation (award no. DGE 2140743).

References

Besançon, M., Papamarkou, T., Anthoff, D., Arslan, A., Byrne, S., Lin, D., & Pearson, J. (2021). Distributions.jl: Definition and modeling of probability distributions in the JuliaStats ecosystem. *Journal of Statistical Software*, *98*(16), 1–30. https://doi.org/10.18637/jss.v098.i16

Colby, D. (2024). *CausalELM.jl*. https://github.com/dscolby/CausalELM.jl

Haneuse, S., & Rotnitzky, A. (2013). Estimation of the effect of interventions that modify the received treatment. *Statistics in Medicine*, *32*(30), 5260–5277. https://doi.org/10.1002/sim.5907

Hernán, M. A., & Robins, J. M. (2020). *Causal inference*. CRC Boca Raton, FL.

Labayle, O., Beentjes, S., Khamseh, A., & Ponting, C. (2024). *TMLE.jl*. https://github.com/olivierlabayle/TMLE.jl

Lin, D., White, J. M., Byrne, S., Bates, D., Noack, A., Pearson, J., Arslan, A., Squire, K., Anthoff, D., Papamarkou, T., Besançon, M., Drugowitsch, J., Schauer, M., & contributors, other. (2019). *JuliaStats/Distributions.jl: a Julia package for probability distributions and associated functions*. https://doi.org/10.5281/zenodo.2647458

Pearl, J. (2009). *Causality*. Cambridge university press.

Quinn, J., Kamiński, B., Anthoff, D., Bouchet-Valat, M., Papp, T. K., Arakaki, T., Schouten, R., Robinson, N., mathieu17g, Samuel, O., Revels, J., ExpandingMan, Hanson, E., Blaom, A., Arslan, A., Ling, J., Chen, J., Day, J., Calderón, J. B. S., … Adenbaum, J. (2024). *JuliaData/Tables.jl: v1.12.0* (Version v1.12.0). Zenodo. https://doi.org/10.5281/zenodo.12753139

Schauer, M., Keller, M., & Wienöbst, M. (2024). *CausalInference.jl*. Zenodo. https://doi.org/10.5281/ZENODO.13684767

DRAFT